Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 14/10/2026**

A Mini Project Report on

Quickzo

An E-Commerce Website Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course **Agile**

Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class **T1**

Submitted by

Name	Roll No.
Harshita Bhatia	2401011
Rashi Bhatia	2401012
Saniya Bijlani	2401015

Under the guidance of

Prof. Sanober Shaikh

Department of **Artificial Intelligence and Data Science,** Thadomal Shahani College of Engineering

2026–2027

CERTIFICATE

This is to certify that the mini project entitled **"Quickzo: An E-Commerce Website Delivered Through an Agile and DevOps Pipeline**" is a bona fide work of

Harshita Bhatia(2401011), Rashi Bhatia(2401012) and **Saniya Bijlani (2401015)** Submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

Prof. Sanober Shaikh	**Dr. Madhuri Rao**
Project Guide	**Head of Department**

Date: _____ Place: _____

ABSTRACT

Modern retail platforms often suffer from high cart abandonment, friction-heavy browsing flows, and opaque fulfillment processes, failing to meet the low-latency demands of the rapidly expanding quick-commerce (Q-Commerce) sector. Shoppers frequently navigate away when confronted with slow interface interactions, complex multi-step checkouts, or a lack of real-time visibility into post-purchase order fulfillment. **Quickzo** addresses these challenges by delivering an intuitive, low-latency, full-stack quick-commerce platform that enables instant category browsing, client-side dynamic cart synchronization, streamlined checkout simulation, and real-time order delivery tracking with step-by-step dispatch status updates.

Developed as the core implementation artifact for the Mini-Project requirement in **Agile Software Development (ASD)**, the platform emphasizes enterprise software engineering best practices, containerization, and modern continuous integration/continuous deployment (CI/CD) workflows. The system architecture decouples a presentation layer served by an **Nginx** reverse proxy and web server from a high-performance **Spring Boot 3.5.6 (Java 21)** backend, backed by an isolated **MySQL 8.0** relational database. The frontend utilizes responsive HTML5, CSS3, and modern JavaScript, leveraging local storage persistence for instantaneous cart computations across diverse inventory verticals including Groceries, Fruits & Vegetables, Skincare, and Stationery. The backend exposes RESTful endpoints (/users, /products, /login, /signup) orchestrated using Spring Data JPA, Hibernate ORM, and layered domain abstractions (Controllers, Services, and Repositories) to ensure transactional integrity and secure authentication.

To satisfy strict quality, reliability, and automated delivery standards, the system incorporates a comprehensive DevOps pipeline implemented through a declarative **Jenkinsfile** and **Docker Compose** orchestration. The automated pipeline enforces environment validation, executes automated Gradle test suites with JUnit report archiving, builds optimized multi-tier container images, deploys containerized microservices within an isolated bridge network, and conducts automated HTTP and TCP smoke tests. Production availability and host readiness are continuously verified through integrated **Nagios Core** service definitions monitoring network ports and API availability. The resulting platform delivers sub-millisecond local interface responsiveness, highly resilient service deployment, and a production-grade blueprint for modern micro-fulfillment web architectures.

Keywords: Agile, Scrum, Git, Docker, Docker Compose, Jenkins, CI/CD, Prometheus, Grafana, Nagios, React, Node.js

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

In modern fast-paced consumer and campus environments, the demand for everyday essentials—such as groceries, fresh produce, personal care items, and stationery—is characterized by urgency and high frequency. Traditional retail outlets require physical transit and queuing at billing counters, while legacy e-commerce platforms operate on delivery cycles spanning several hours or days, failing to address immediate, on-demand necessities. Quick commerce (Q-Commerce) has emerged as an indispensable paradigm to bridge this gap, leveraging hyper-local micro-fulfillment, real-time inventory visibility, instant digital cart management, and rapid dispatch workflows.

The Agile Software Development (ASD) & DevOps laboratory introduced, across discrete experimental modules, the fundamental engineering practices required to build and deliver modern software systems reliably: distributed version control (Git), containerization (Docker), multi-container orchestration (Docker Compose), continuous integration and continuous delivery (Jenkins CI/CD), reverse proxy routing (Nginx), and infrastructure observability (Nagios). Each laboratory exercise was performed in isolation on simplified, individual workloads.

This mini-project synthesizes all these principles into a unified, realistic product—Quickzo: A Quick Commerce Web Application. By engineering a complete three-tier architecture (responsive web frontend, Spring Boot backend, and MySQL database) wrapped in an automated deployment and monitoring pipeline, the interaction, dependencies, and operational lifecycle of these tools become fully demonstrable within a cohesive, production-like environment.

1.2 Problem Statement

To design, develop, containerize, and operate a full-stack quick-commerce web application (**Quickzo**) that enables consumers to effortlessly browse categorized essential goods, manage dynamic shopping carts with zero reload latency, execute checkout workflows, and track simulated rider delivery in real time; and to deliver the entire system through an Agile, fully automated DevOps lifecycle wherein application changes are version-controlled in Git, tested and built via a declarative Jenkins pipeline, orchestrated as isolated micro-containers via Docker Compose, and continuously supervised through Nagios infrastructure monitoring.

1.3 Objectives

1. User Authentication & Account Management: Develop secure registration and login mechanisms allowing users to create accounts, authenticate credentials, and persist user data reliably via Spring Boot REST APIs and MySQL.
2. Dynamic Multi-Category Catalog Discovery: Implement an intuitive browsing interface across multiple essential categories (*Groceries, Fruits & Vegetables, Skincare, Stationery*), allowing customers to view product inventory and pricing.
3. Real-Time Client-Side Cart Management: Enable instantaneous product addition, count adjustments, price recalculations, and item removals utilizing browser localStorage to guarantee zero page-reload lag.
4. Checkout Simulation & Order Settlement: Provide a frictionless checkout and billing interface that summarizes order totals and simulates payment completion.
5. Interactive Rider Tracking & Status Progression: Build an order tracking module (rider-info) simulating end-to-end delivery phases (*Order Picked*, *Out for Delivery*, *Delivered*) with dynamic progress indicators, estimated time of arrival (ETA) countdowns, and cancellation capabilities.
6. Multi-Container Architecture & Orchestration: Containerize the Nginx web server/reverse proxy, Spring Boot application, and MySQL database using Docker, enabling the complete stack to be spun up or down with a single docker compose command using persistent data volumes and health-check dependencies.
7. Automated CI/CD Pipeline with Jenkins: Construct an end-to-end declarative pipeline (Jenkinsfile) that automates code checkout, environment verification, backend unit testing with Gradle, JUnit report generation, Docker image compilation, and automated stack deployment.
8. Automated Health & Smoke Verification: Implement automated smoke tests within the continuous delivery pipeline to programmatically validate HTTP 200 responses across frontend (port 80) and backend API endpoints (/products) before certifying deployment success.
9. Infrastructure Observability & Service Monitoring: Configure Nagios Core (quickzo-nagios.cfg) to continuously monitor host availability, TCP port status, and endpoint health across the frontend (port 80), backend (port 8081), and database (port 3306) services.

1.4 Scope

The system targets a hyper-local quick-commerce model covering daily retail categories: Groceries, Produce, Skincare, and Stationery. Third-party payment gateways and live banking transactions are out of scope; the application provides a simulated checkout and payment interface. Real-time GPS telemetry and external SMS notifications are out of scope, with delivery progress and ETAs rendered directly via the browser-based rider tracking page. The deployment target is a single host running Docker Engine; the same Docker Compose configuration, Jenkins CI/CD pipeline, and Nagios monitoring apply directly to a college lab machine or cloud VM.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in CampusEats	Section
1	Version control with Git	Branch per backlog issue, 15 pull requests, merge commits, release tag v1.0	5.1
2	Containerisation with Docker	Dockerfiles for the API (non-root, health check) and the frontend (multi-stage, Nginx)	5.2
3	Docker Compose	Seven-service stack with health-gated start-up, named volumes, one network	5.3
4	Jenkins CI pipeline	Multibranch declarative pipeline: install, lint, test (JUnit), build images	5.4
5	Automated deployment	Deploy on main, version-aware smoke test, automatic rollback	5.5
6	Agile project lifecycle	Scrum project CE, 2 sprints, story points, issue keys in branches, commits and builds	2
7	Monitoring	Prometheus + Grafana for metrics, Nagios + NRPE for availability	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Student	Discover essential daily goods, manage cart with instant updates, complete checkout, and track delivery progress in real time.	Home Catalog (Quickzo.html), Category Pages (Groceries.html, Fruits&Vegetables.html, etc.), Cart (cartpage.html), Payment (pay.html), Rider Tracking (rider-info.html), Authentication (login.html, signup.html)
Kitchen staff	Receive assigned customer deliveries, view destination ETAs, update delivery milestones, and handle delivery cancellations.	Rider Dispatch & Live Tracking Page (rider-info.html)
Administrator	Maintain product catalog stock via REST APIs, manage user accounts, oversee automated deployments, and monitor system health.	Product & User API Endpoints (/products, /users), Jenkins CI/CD Console, Nagios Monitoring Dashboard

Table 2.1: User roles

2.2 Functional Requirements

- User Authentication: Customers register and log in with a username and password; credentials are submitted asynchronously to the backend (/users/signup, /users/login) and verified against the MySQL users database.
- Categorized Catalog Discovery: The product catalog is organized into dedicated categories (*Groceries, Fruits & Vegetables, Skincare, Stationery*), allowing customers to browse items with clear images, names, and pricing.
- Dynamic Client-Side Cart Management: Customers can add items to the cart, dynamically increment or decrement item quantities via an interactive counter (+ / -), automatically remove items when the quantity reaches zero, and persist their cart across page reloads using browser localStorage.
- Cart Summary & Subtotal Calculation: The cart page (cartpage.html) dynamically aggregates all selected items, computes item-level subtotals, and updates the final payable order amount in real time.
- Simulated Checkout & Payment: Customers transition from cart review to a streamlined payment page (pay.html) to review delivery details and simulate payment completion.
- Live Rider Dispatch & ETA Tracking: The order tracking interface (rider-info.html) simulates real-time fulfillment across sequential stages: Order Picked (ETA 20 min, 30% progress), Out for Delivery (ETA 10 min, 70% progress), Delivered (ETA 0 min, 100% progress).
- Delivery Cancellation: Customers can cancel an active order prior to final delivery directly from the tracking screen, instantly halting progress updates, resetting the ETA, and updating order status.
- Product Inventory Management (CRUD): The Spring Boot backend exposes REST endpoints (/products) enabling administrators to retrieve, create, update, and delete product catalog entries and monitor stock quantities.

2.3 Non-Functional Requirements

- Performance: Zero-latency cart updates and instant catalog browsing powered by client-side caching (localStorage); Nginx serves static assets with Gzip compression and a 1-day client cache policy (expires 1d).
- Security: Reverse proxy masking prevents direct public exposure of internal Spring Boot endpoints; database credentials are fully externalized via environment variables and Jenkins Credentials Store rather than hardcoded in source control.
- Reliability & Fault Tolerance: Container health checks ensure strict startup ordering—the Spring Boot backend container initializes only after MySQL reports a healthy state (mysqladmin ping). All containers use the restart: unless-stopped policy, with data preserved in a dedicated Docker named volume (quickzo_mysql_data).
- Portability: The multi-tier architecture (Nginx, Spring Boot, MySQL) starts reliably on any system running Docker Engine using a single docker compose up -d command over an isolated bridge network (quickzo-network).
- Continuous Integration & Delivery: Every build is automated through Jenkins, compiling Java 21 sources, running Gradle unit tests with JUnit report publishing, building Docker images, deploying containers, and executing automated smoke tests.

- Observability: Service availability and port readiness across Frontend (port 80), Backend API (port 8081), and MySQL (port 3306) are actively monitored using Nagios Core (quickzo-nagios.cfg).

2.4 Scrum Setup

The project was organized using the Scrum framework, adopting the Agile project lifecycle practiced in the laboratory. Every requirement was captured as a backlog issue identified by a unique key from **QZ-1** to **QZ-15**, assigned an estimate, an owner, and a target sprint. Issue keys were embedded into Git branch names (e.g., feature/QZ-1-user-auth), commit messages, Pull Request titles, and Jenkins build descriptions, providing full traceability from requirement definition to production deployment.
User-facing capabilities were structured as user stories following the format:
"As a <role>, I want <capability>, so that <benefit>" with defined acceptance criteria, and estimated using Fibonacci story points (3, 5).
Technical foundation tasks—such as establishing repositories, crafting Dockerfiles, configuring multi-container Compose stacks, building Jenkins CI/CD pipelines, and setting up Nagios monitoring—were created as unpointed engineering tasks. Deficiencies uncovered during pipeline execution were logged as bugs. Table 2.2 outlines the complete product backlog.

2.5 Product Backlog

Key	Type	Summary	Pts	Owner	Sprint
QZ-1	Story	User registration and authentication via Spring Boot REST API	5	Member 1	1
QZ-2	Story	Multi-category product catalog browsing (Groceries, Produce, Skincare, Stationery)	3	Member 2	1
QZ-3	Story	Dynamic client-side cart management with quantity counter and localStorage	5	Member 3	1
QZ-4	Story	Cart review page with real-time total calculations and item removal	3	Member 2	1
QZ-5	Task	GitHub repository setup, branching model, and .gitignore configuration	—	Member 1	1
QZ-6	Task	Multi-tier Dockerfiles (Nginx & Spring Boot) and Docker Compose stack configuration	—	Member 3	1
QZ-7	Task	Declarative Jenkins CI pipeline: workspace validation, Gradle test execution, JAR build	—	Member 1	1
QZ-8	Story	Checkout simulation and payment confirmation workflow	3	Member 3	1
QZ-1	Story	User registration and authentication via Spring Boot REST API	5	Member 1	1
QZ-2	Story	Multi-category product catalog browsing (Groceries, Produce, Skincare, Stationery)	3	Member 2	1
QZ-9	Story	Real-time rider tracking screen with automated ETA countdown and progress bar	5	Member 2	2
QZ-10	Story	In-transit order cancellation mechanism with alert confirmation	3	Member 3	2
QZ-11	Story	Spring Boot RESTful Product CRUD API (/products) for inventory management	5	Member 1	2

QZ-12	Task	Jenkins CD pipeline: credential management, automated deployment, and container rollout	—	Member 2	2
QZ-13	Task	Automated pipeline smoke tests validating HTTP 200 on port 80 and /products API	—	Member 1	2

Table 2.2: Product backlog of project CE

2.6 Sprint Plan

Sprint	Goal	Stories (points)	Tasks
QZ Sprint 1	Customers can register/login, browse multi-category products, manage dynamic carts, and complete simulated checkout; the multi-tier stack is containerized with Docker and automatically tested and built by Jenkins CI on every commit.	QZ-1 (5), QZ-2 (3), QZ-3 (5), QZ-4 (3), QZ-8 (3) = 19	QZ-5, QZ-6, QZ-7
Sprint	**Goal**	**Stories (points)**	**Tasks**
QZ Sprint 2	Customers can track live rider delivery progression and cancel orders; store administrators can manage product inventory via REST APIs; every merge to master is deployed automatically with smoke tests, and the entire stack is monitored via Nagios.	QZ-9 (5), QZ-10 (3), QZ-11 (5)= 13	QZ-12, QZ-13, QZ-14, Bug QZ-15

Table 2.3: Sprint goals and scope

Member	Stories	Points	Tasks / bugs
Member 1	QZ-1 (User Auth), QZ-11 (Product CRUD API)	10	QZ-5 (Git setup), QZ-7 (Jenkins CI), QZ-13 (Smoke testing)
Member 2	QZ-2 (Category Catalog), QZ-4 (Cart Summary), QZ-9 (Rider Live Tracking)	11	QZ-12 (Jenkins CD Deployment), QZ-14 (Nagios Monitoring)
Member 3	QZ-3 (Dynamic Cart & LocalStorage), QZ-8 (Checkout Flow), QZ-10 (Order Cancellation)	11	QZ-6 (Docker Compose Stack), QZ-15 (Bug: DB Startup Race Condition)
Total	8 stories	32	6 tasks, 1 bug

Table 2.4: Distribution of work across the team

2.7 Definition of Done:

An issue is moved to **Done** on the Agile board only when all of the following conditions hold:

1. **Traceable Branching & Commit Conventions**: Every commit for the issue explicitly references its issue key in the commit message, and feature branches follow the naming pattern feature/QZ-<n>-<slug> (or fix/QZ-<n>-<slug> for defect resolutions).
2. **Pull Request Quality Gates**: The proposed changes reach the master branch exclusively through a GitHub Pull Request where the automated Jenkins status check has passed with a green build.

3. **Automated Pipeline Deployment & Smoke Test Verification**: The Jenkins build of master containing the merged commit reports **SUCCESS**, which verifies that:
    - Backend Gradle compilation and unit tests passed, and JUnit reports were published.
    - Multi-container Docker images built cleanly.
    - Containers were deployed using docker compose up -d.
    - Post-deployment smoke tests confirmed HTTP 200 OK responses on the frontend root (/) and the backend API endpoint (/products).
4. **End-to-End Functional Acceptance**: The newly introduced feature (e.g., dynamic cart modification, authentication, or rider status updates) has been manually validated in the running live application.

The third condition establishes the direct link between project planning and the DevOps pipeline: an issue cannot be closed on the sprint board while the build that delivers it to the environment is failing.

3. System Design

3.1 Architecture

Quickzo is engineered as a decoupled, containerized three-tier web application orchestrated via Docker Compose, as shown in Figure 3.1.

1. **Presentation & Reverse Proxy Tier (Nginx)**: The client browser loads a responsive, lightweight web application served directly by an **Nginx** web server container on port 80. Nginx delivers static assets (.html, .css, .js, media assets) with Gzip compression and browser caching. Critically, Nginx functions as a **reverse proxy**: all client API requests targeting /login, /signup, /users, /products, and /api/ are forwarded internally to the backend service (http://backend:8081). This eliminates Cross-Origin Resource Sharing (CORS) complexity in production by allowing the browser to interact with a single unified origin (http://localhost:80). Client-side state, such as cart items, count increments, and subtotals, is managed locally using browser localStorage to ensure instant responsiveness without unnecessary server roundtrips.
2. **Application & Business Logic Tier (Spring Boot)**: The backend is an asynchronous, enterprise-grade REST service built on **Spring Boot 3.5.6 (Java 21)** running inside a dedicated container on port 8081. It follows a layered architectural design pattern consisting of **Controllers** (UserController, ProductController), **Services** (UserService, ProductService), **Repositories** (UserRepository, ProductRepository), and **Entity Models** (User, Product). Object-relational mapping and database transactions are abstracted via Spring Data JPA and Hibernate.
3. **Data Tier (MySQL 8.0)**: Relational data—including user credentials and catalog inventory across groceries, produce, skincare, and stationery—is persisted in a containerized **MySQL 8.0** database instance. The database runs on internal port 3306 (with port 3307 exposed externally for administrative tools) and mounts a persistent Docker named volume (quickzo_mysql_data) to prevent data loss across container teardowns and redeployments. Database schemas and initial catalog seed entries are initialized on first boot via quickzo_mysql_migration.sql.
4. **DevOps CI/CD & Monitoring Ecosystem**:
    - **Jenkins CI/CD Automation**: Operating outside the production network, Jenkins tracks GitHub commits, executes ./gradlew clean test bootJar, archives test reports, builds updated Docker images, manages environment credentials, and executes container deployments and automated HTTP smoke tests.
    - **Nagios Core Infrastructure Observability**: Configured alongside the application (quickzo-nagios.cfg), Nagios continuously monitors host liveness and performs

scheduled HTTP health checks against the Nginx frontend (port 80), TCP checks against the Spring Boot API (port 8081), and TCP checks against the MySQL database (port 3306).

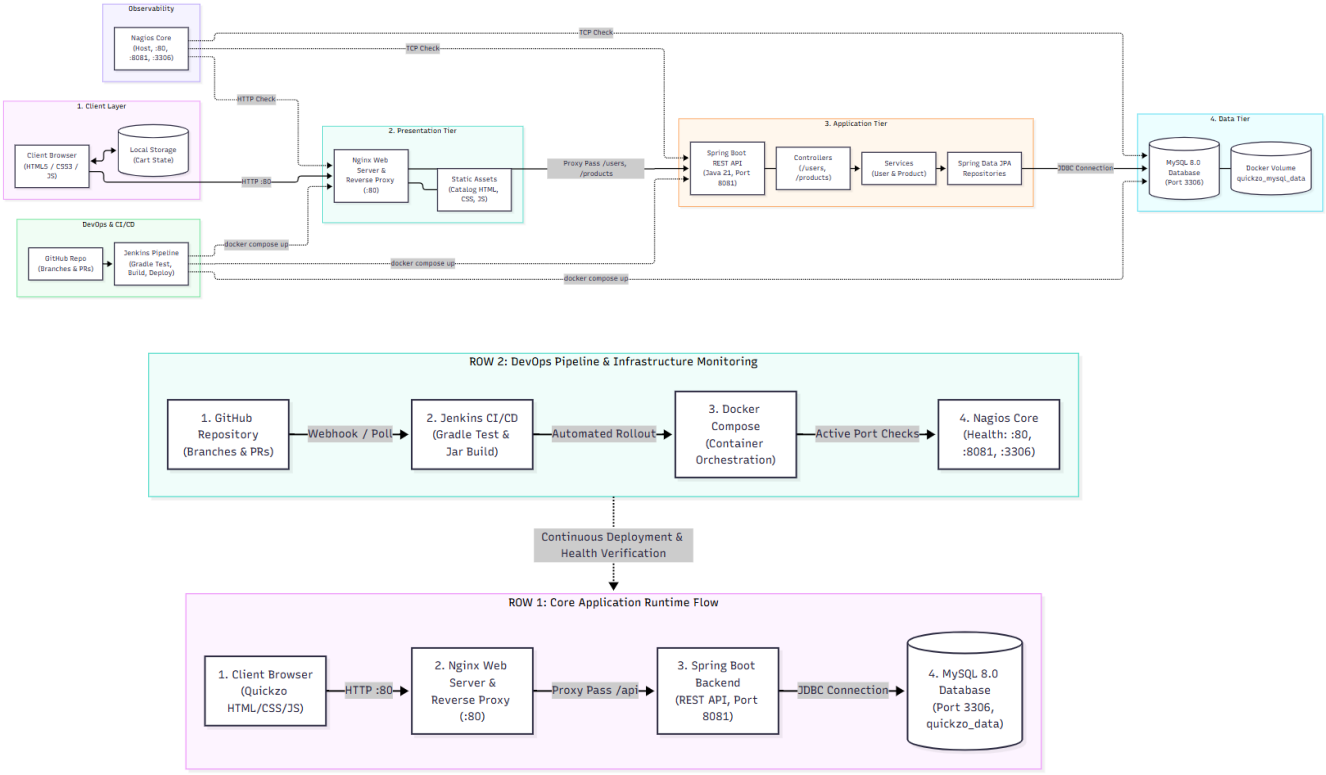


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+), Web Storage API (localStorage)	Ultra-lightweight with zero build overhead; native DOM manipulation enables instantaneous cart counter updates and persistent offline cart caching without framework bloat.
Web Server & Reverse Proxy	Nginx (Alpine Linux)	High-performance static asset delivery with Gzip compression and browser caching (expires 1d); proxies /login, /signup, /users, and /products to the backend to eliminate CORS issues in production.
Layer	**Technology**	**Reason for choice**
API / Backend	Java 21, Spring Boot 3.5.6 (Spring Web MVC, Spring Data JPA, Hibernate)	Strongly-typed, enterprise-grade REST architecture; clean layered separation (Controller $\rightarrow$ Service $\rightarrow$ Repository); built-in connection pooling and robust object-relational mapping.
Authentication & User Management	RESTful Auth Service (/users/signup, /users/login), Spring MVC	Lightweight asynchronous authentication and user record persistence validated against MySQL with clean JSON request/response handling.
Database	MySQL 8.0 (InnoDB, utf8mb4)	ACID-compliant relational storage maintaining strict integrity between user accounts and multi-category catalog stock; automated initialization via SQL migration script (quickzo_mysql_migration.sql).

Containerization & Orchestration	Docker, Docker Compose (compose.yaml)	Encapsulates frontend, backend, and database into isolated containers on a dedicated bridge network (quickzo-network); enforces health-check-guided startup ordering (mysqladmin ping).
Build & Testing	Gradle Wrapper 8.14.3, JUnit 5 Platform	Reproducible, wrapper-driven builds (./gradlew clean test bootJar); executes backend test suites and outputs standard XML reports for automated archiving.
CI/CD Automation	Jenkins Declarative Pipeline (Jenkinsfile)	Version-controlled pipeline automating checkout, prerequisite validation, test execution, JUnit reporting, Docker image building, credential injection, and post-deployment smoke tests.
Infrastructure Monitoring	Nagios Core (quickzo-nagios.cfg)	Active, state-based availability checks continuously verifying host uptime and port connectivity across Frontend (port 80), Backend API (port 8081), and MySQL (port 3306).

Table 3.1: Technology stack

3.3 Database Design

The relational schema in **MySQL 8.0** is structured to guarantee transactional integrity between user authentication, catalog inventory, and ordering workflows (Figure 3.2).

1. **users Table**: Stores authenticated customer credentials (id, username, password).
   The username column is enforced with a UNIQUE constraint and indexed to allow constant-time lookups during login validation (/users/login) and prevent duplicate registrations (/users/signup).
2. **product Table**: Maintains the core catalog inventory across the four retail verticals (Groceries, Fruits & Vegetables, Skincare, and Stationery). Each row contains a primary key (id), product name, and an in-stock quantity. Catalog changes performed through administrative REST endpoints (/products) update this centralized inventory store.
3. **Cart & Order Persistence**: In the client layer, the active shopping cart state is captured dynamically in browser localStorage containing product identifiers, item name, snapshot price, and count. When advancing through checkout (pay.html) and rider tracking (rider-info.html), order line items preserve the item name and unit price at the time of purchase. This snapshotting mechanism ensures that any subsequent price modifications or inventory count adjustments in the product catalog never alter the billing records of completed orders.
4. **Data Reliability & Migrations**: Schema creation and initial seeding of 32 multi-category items are automated via quickzo_mysql_migration.sql executed during container startup. The database character set is standardized to utf8mb4 with utf8mb4_unicode_ci collation, and transactional data is safeguarded across container restarts using a persistent Docker named volume (quickzo_mysql_data).

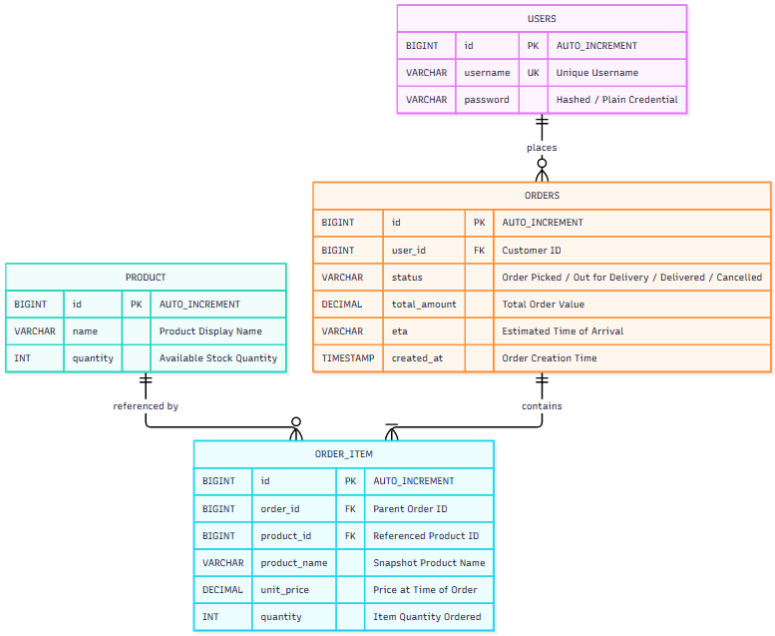


Figure 3.2: Entity-relationship diagram

3.4 Order Lifecycle

The order state machine (Figure 3.3) enforces a strict progression through ORDER PICKED OUT FOR DELIVERY, DELIVERED, updating the ETA and progress bar at each step. Disallowed transitions are blocked once an order enters a completed state. A customer may cancel the delivery at any point before arrival, which immediately transitions the order to CANCELLED BY USER and halts further progress.

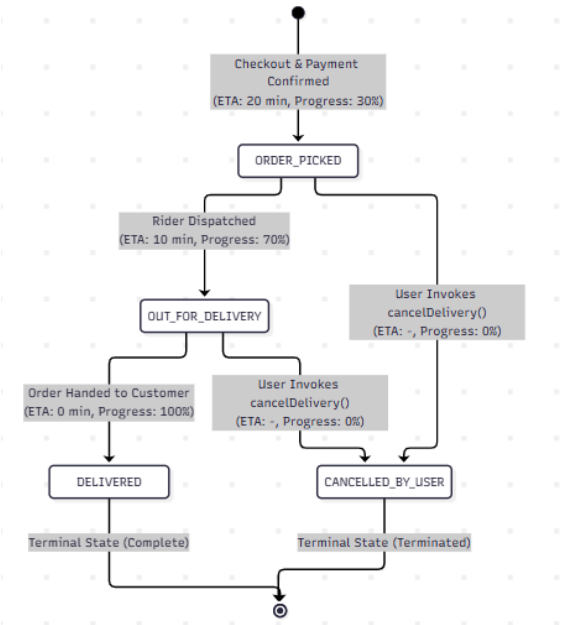


Figure 3.3: Order state machine

3.5 API Design

Method	Endpoint	Role	Purpose
POST	/login or /users/login	Public	Authenticate user credentials and grant access
GET	/users or /users/all	Admin	Retrieve the list of all registered customer usernames
GET	/products	Public	Fetch the complete catalog and stock quantities across all categories
GET	/products/{id}	Public	Retrieve detailed product information by unique ID
POST	/products	Admin	Add a new product and stock entry to the catalog
PUT	/products/{id}	Admin	Update existing product details, pricing, or inventory quantity
DELETE	/products/{id}	Admin	Remove a product from the inventory database
GET	/ (Port 80)	Public	Serve the main catalog interface and acts as frontend health check
GET	/products (Port 8081)	Internal / CI	Direct database connectivity check and smoke test endpoint

Table 3.2: REST API

3.6 Security Design

- Reverse Proxy Shielding: Public clients only communicate with the Nginx reverse proxy on port 80. The Spring Boot application (port 8081) and MySQL database (port 3306) are isolated within the internal Docker bridge network (quickzo-network).
- Secret & Credential Externalization: Database passwords and root secrets are never committed to version control; .env and sensitive configurations are ignored by Git and injected dynamically at deployment time via Jenkins Credentials Store.
- SQL Injection Prevention: All persistence operations use Spring Data JPA and Hibernate ORM, ensuring database queries are executed via parameterized prepared statements.
- CORS Mitigation: Nginx acts as a unified reverse proxy routing both frontend static pages and /users or /products API requests under the same origin, eliminating cross-origin browser vulnerabilities in production.
- Payload & Input Validation: Incoming registration and catalog modification requests are validated through the Spring Boot validation tier to prevent malformed data insertion.

4. Application Implementation

The backend is organized into a clean, layered Spring Boot architecture consisting of **Controllers** (ProductController, UserController), **Services** (ProductService, UserService), and **Repositories** (ProductRepository, UserRepository). Database interactions are abstracted using Spring Data JPA interfaces extending JpaRepository, enabling seamless CRUD operations and transactions without boilerplate SQL. Automated Gradle builds compile the source code, run unit tests, and package the application into a standalone executable JAR (bootJar) within seconds. The frontend is built as a lightweight, modular web interface served by Nginx, utilizing native DOM APIs and browser localStorage for zero-latency client state management.

4.1 Customer Experience

After registering and logging in (Figure 4.1), customers access the main platform to browse essential goods across categorized catalogs—Groceries, Fruits & Vegetables, Skincare, and Stationery (Figure 4.2). Products can be added to the cart instantly using an interactive counter (+ / -) that updates quantities without page reload. The cart page (Figure 4.3) aggregates all selected items from localStorage, computing real-time subtotals and enabling one-click removals before routing the customer to the simulated payment gateway (pay.html). Following order placement, the live tracking screen (Figure 4.4) simulates delivery progression across three sequential milestones (*Order Picked*, *Out for Delivery*, *Delivered*), updating ETAs and dynamic progress bars every five seconds while allowing customers to cancel an active in-transit delivery.

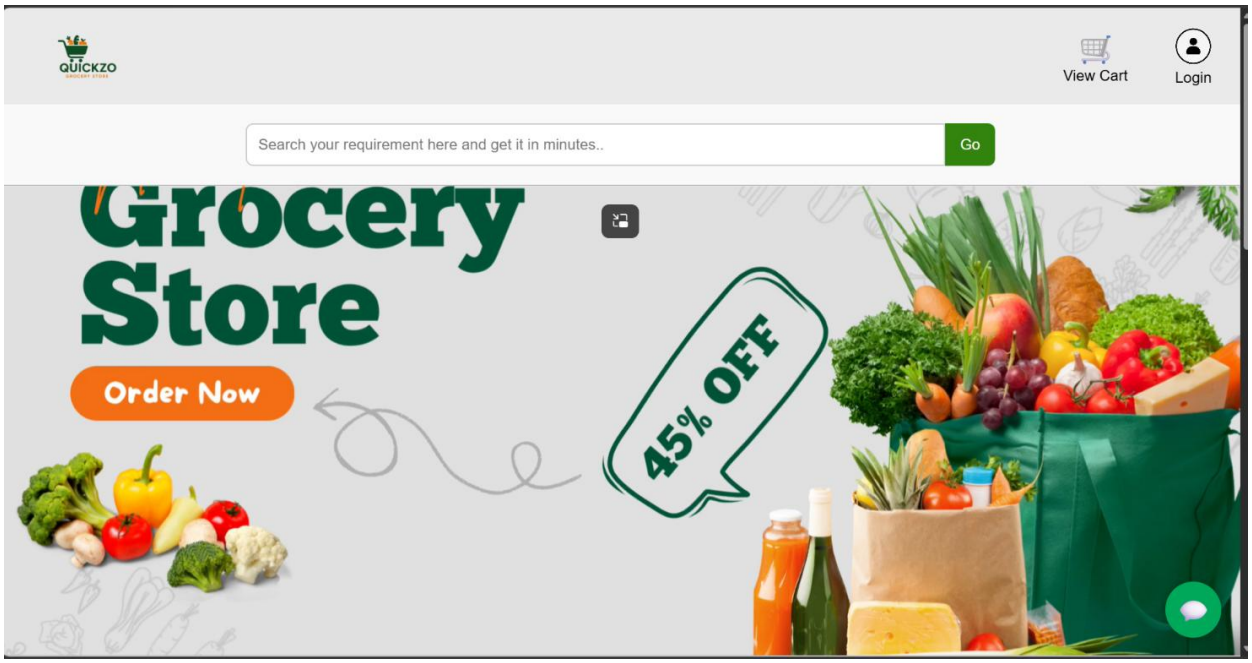


Figure 4.1: Login page with demo-account shortcuts

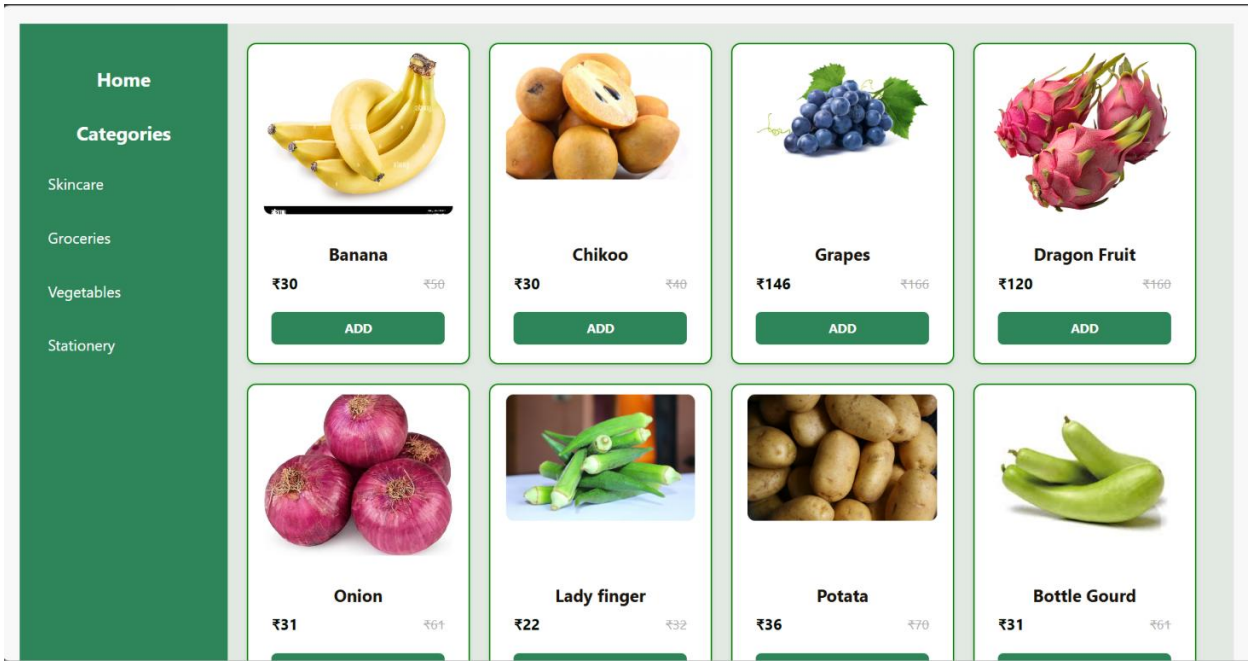


Figure 4.2: Menu with category filter and cart

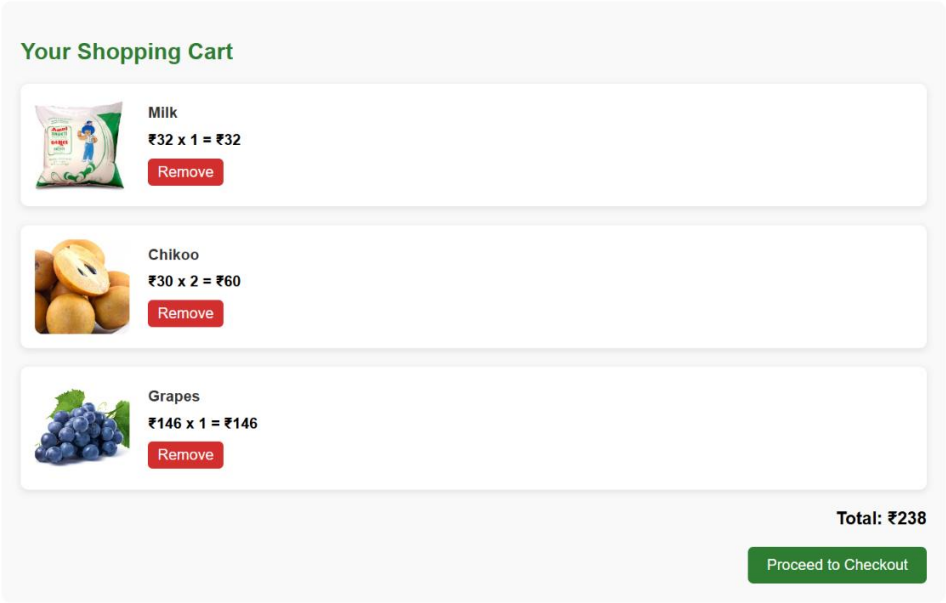


Figure 4.3: Cart visuals

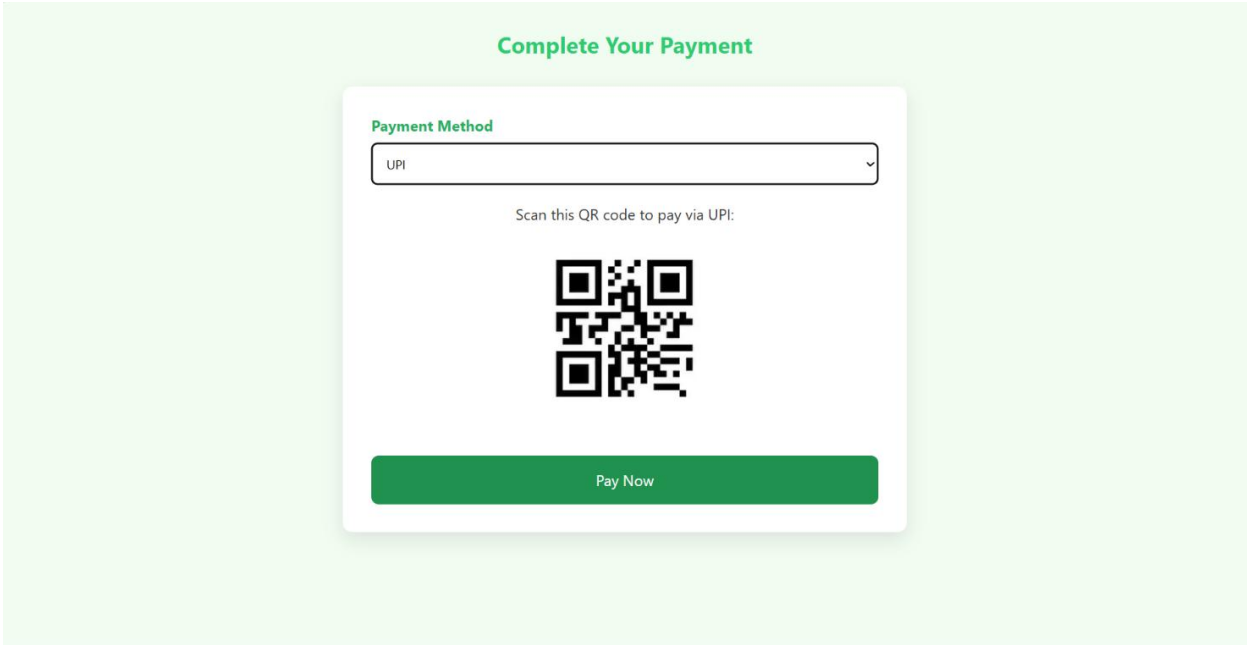


Figure 4.4: Payment page

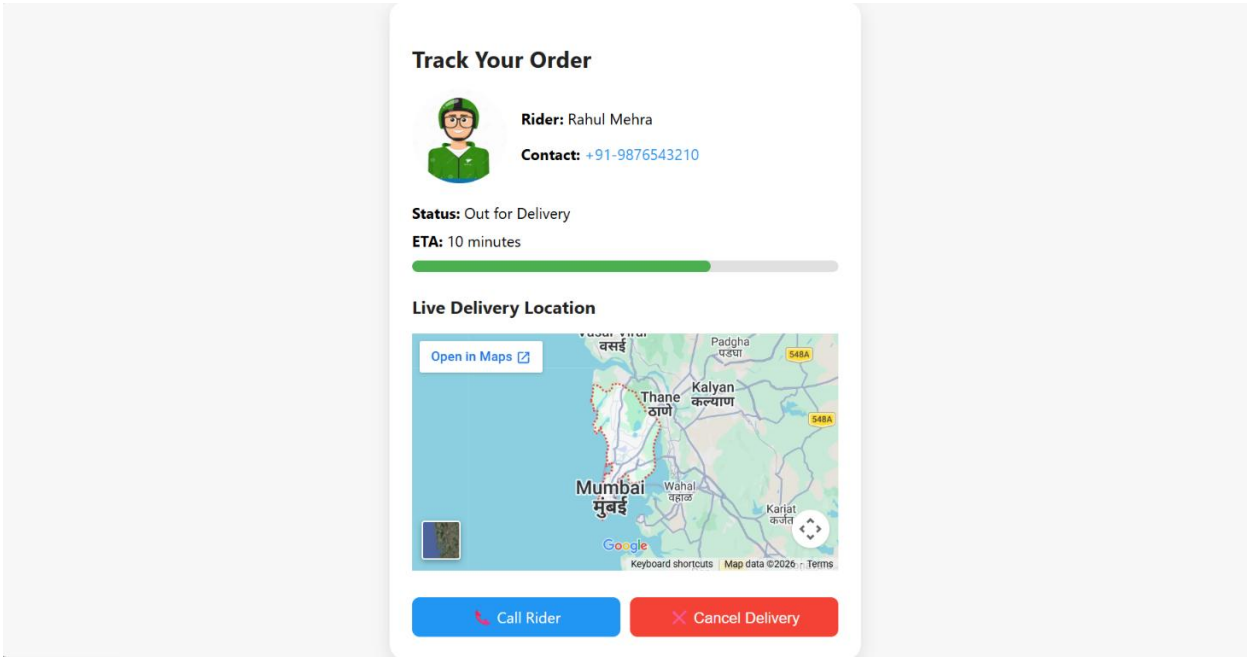


Figure 4.5: Live Order Tracking

4.2 Order Fulfillment & Rider Tracking

Fulfillment and delivery progression are visualized on the live dispatch board (Figure 4.5), which automatically updates across three sequential phases: **Order Picked**, **Out for Delivery**, and **Delivered**. The tracking interface displays the assigned delivery partner details, an active ETA countdown (20 min 10 min 0 min), and a dynamic progress bar (30%, 70% , 100%) advancing at five-second intervals. Customers retain the ability to halt an active delivery using the cancellation action, which resets the progress indicator and marks the order cancelled. On the inventory side, store administrators adjust stock levels using the RESTful product API (PUT /products/{id}), immediately updating product availability across the catalog. Figure 4.5 was captured during an active delivery simulation.

4.3 Administrator Tools

The administrator suite provides centralized catalog governance and operational oversight. Inventory management enables administrators to execute full CRUD operations over the product database via dedicated REST endpoints (/products), allowing them to add new SKUs, modify stock levels, adjust unit prices, and remove depleted items across all four retail categories. User oversight interfaces with UserController (/users/all) to audit registered customer accounts. Infrastructure and operational health including container uptime, service readiness on ports 80, 8081, and 3306, and deployment execution logs are monitored through the integrated **Nagios Core** dashboard and **Jenkins** build history, providing administrators with continuous visibility into system stability and performance. 0

4.4 Caching and Token Generation

Catalog performance is optimized at the edge through Nginx, which enforces an active client caching policy (expires 1d, Cache-Control "public, no-transform") alongside Gzip compression for all HTML, CSS, JavaScript, and media assets, eliminating redundant network latency during repeated category navigation. Shopping cart state is cached client-side using browser localStorage; counter increments and deletions update the serialized local store atomically, ensuring real-time subtotal synchronization across pages without server overhead. On the backend, database uniqueness and record identity are governed by MySQL InnoDB AUTO_INCREMENT primary keys and unique column constraints, ensuring thread-safe identifier generation and preventing concurrent registration collisions.

4.5 Service Health Verification and Observability

Service availability and infrastructure health in Quickzo are instrumented through active health checks in Docker, automated Jenkins pipeline probes, and Nagios Core service definitions (quickzo-nagios.cfg). Nagios enforces continuous monitoring across all three containers using 24x7 check periods and three-attempt retry thresholds: host reachability is verified via check-host-alive, frontend web availability is audited via HTTP port 80 probes, and backend application and database liveness are verified via TCP socket checks on ports 8081 and 3306, respectively. At the container level, the MySQL service runs scheduled mysqladmin ping health checks, while the Jenkins deployment pipeline instruments automated curl-based smoke tests that evaluate HTTP 200 OK responses on the root (/) and /products API endpoints before certifying release readiness.

5. DevOps Implementation

This chapter describes how each Agile and DevOps laboratory experiment was integrated into **Quickzo**. Figure 5.1 summarizes the automated continuous delivery pipeline that binds them together: a development task originates on a Git branch named after its backlog issue (feature/QZ-<n>-<slug>), is automatically verified, tested, and compiled by Jenkins, merges into master via pull request, and is subsequently deployed as containerized services via Docker Compose, validated through automated HTTP smoke tests, and continuously supervised by Nagios monitoring.

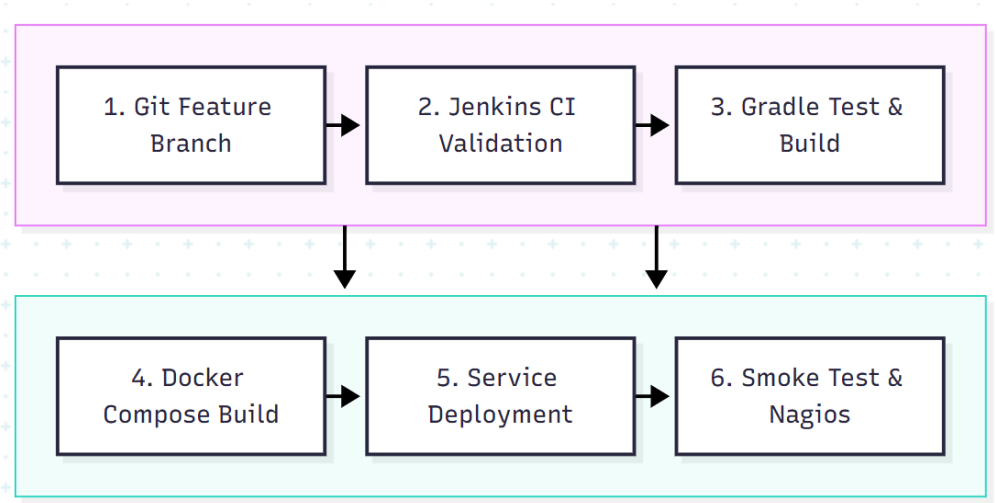


Figure 5.1: Delivery pipeline from push to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

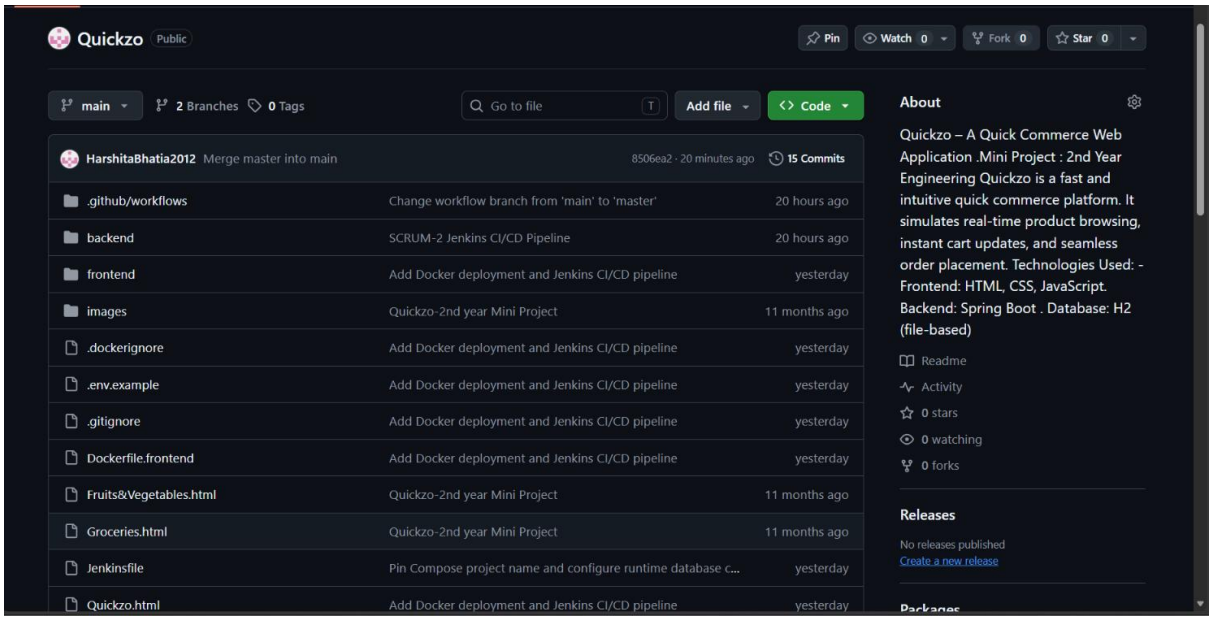
The repository HarshitaBhatia2012/Quickzo is hosted on GitHub. The branching strategy established in issue **QZ-5** enforces an isolated branch per backlog issue: named feature/QZ-<n>-<slug> for user stories and engineering tasks, and fix/QZ-<n>-<slug> for defect resolutions. Every commit message begins with its associated issue key (e.g., QZ-1: Add user authentication endpoints).
Changes reach the master branch exclusively through GitHub Pull Requests merged with explicit merge commits, ensuring each feature's commit sequence remains grouped and traceable in the commit history. The commit author reflects the team member who owned the task, while the pull request review and merge are executed against master.
account.

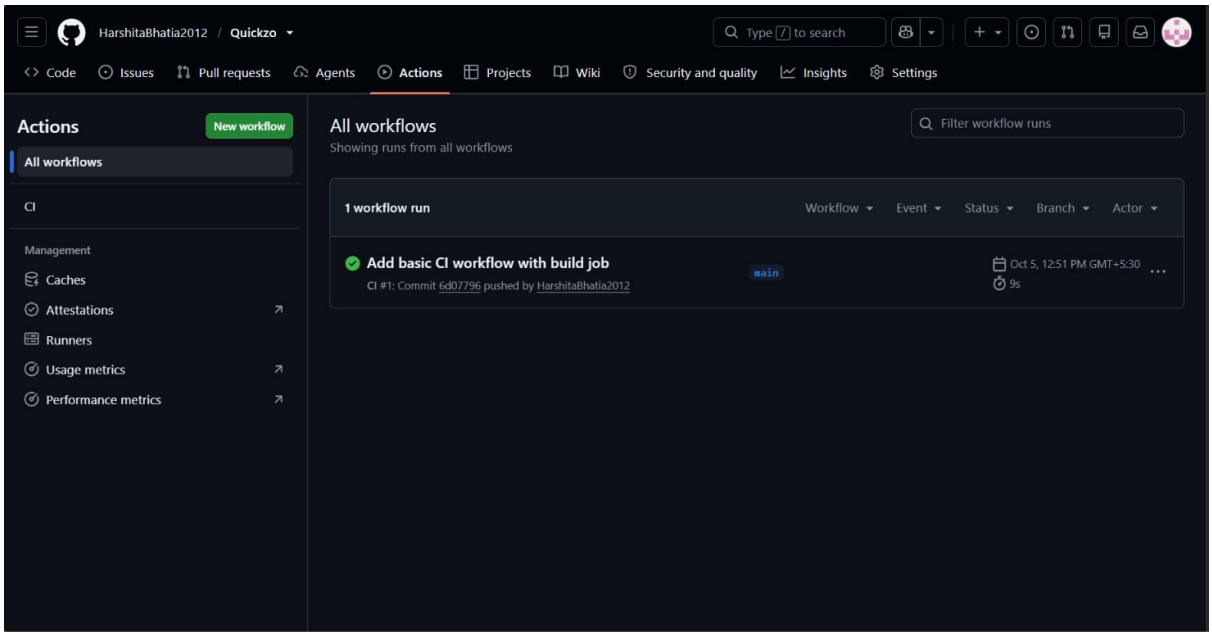
PR	Title	Owner
#1	QZ-1 User registration and authentication via Spring Boot REST API	Member 1
#2	QZ-2 Multi-category product catalog browsing (Groceries, Produce, etc.)	Member 2
#3	QZ-3 Dynamic client-side cart management with localStorage	Member 3
#4	QZ-4 Cart review page with real-time total calculations	Member 2
#5	QZ-8 Checkout simulation and payment confirmation workflow	Member 3
#6	QZ-6 Multi-tier Dockerfiles (Nginx & Spring Boot) and Compose stack	Member 3
#7	QZ-7 Declarative Jenkins CI pipeline: Gradle test & bootJar packaging	Member 1
#8	QZ-9 Real-time rider tracking screen with automated ETA countdown	Member 2
#9	QZ-10 In-transit order cancellation mechanism with alert confirmation	Member 3
#10	QZ-11 Spring Boot RESTful Product CRUD API (/products)	Member 1
#11	QZ-12 Jenkins CD pipeline: automated Docker Compose deployment	Member 2
#12	QZ-13 Automated pipeline smoke tests validating HTTP 200 responses	Member 1
#13	QZ-14 Nagios Core active monitoring configuration for the stack	Member 2
#14	QZ-5 Repository setup, .gitignore, and project documentation	Member 1
#15	QZ-15 Fix backend connection failure during database initialization	Member 3

Table 5.1: Pull requests merged into main The resulting history, abbreviated, shows each issue's commits on its own branch followed by the merge:

```
$ git log --oneline --graph
* f19c6b8 Merge pull request #15 from fix/QZ-15-db-healthcheck-wait
|\
| * e39ed34 QZ-15: Enforce depends_on service_healthy for MySQL in compose.yaml
|/
* e93384e Merge pull request #14 from feature/QZ-5-repo-readme
* ac8168f Merge pull request #13 from feature/QZ-14-nagios-monitoring
* 24915a8 Merge pull request #12 from feature/QZ-13-automated-smoke-tests
...
* 5f97de3 Merge pull request #7 from feature/QZ-7-jenkins-ci-pipeline
|\
| * ca91510 QZ-7: Archive JUnit test results from backend Gradle wrapper
| * 358f3b6 QZ-7: Configure declarative Jenkinsfile with multi-stage build
|/
...
```

At the conclusion of Sprint 2, the commit at the head of master was tagged v1.0 and published as a GitHub release. Should a production release ever need to be rolled back, executing a git revert on its merge commit creates a new forward commit that cleanly reverses the changes without rewriting Git history—the non-destructive rollback strategy practiced in Experiment 1—whereupon the Jenkins pipeline automatically deploys the reverted state.





5.2 Containerisation with Docker (Experiment 2)

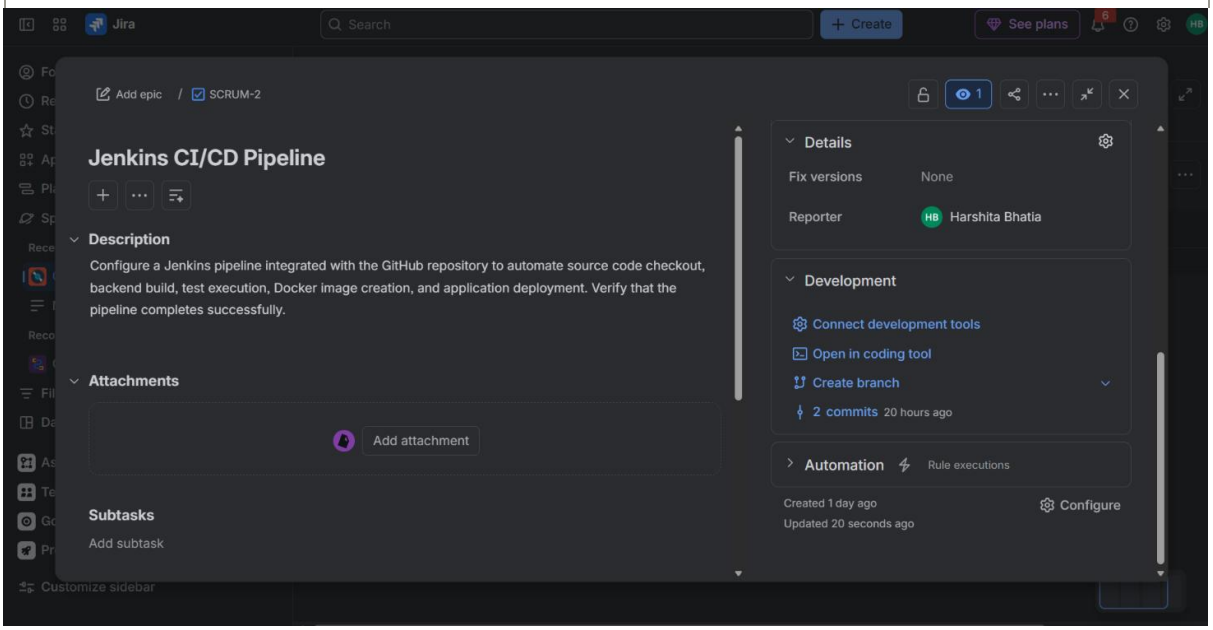
Each tier in **Quickzo** has a dedicated Dockerfile optimized for build efficiency, small image footprints, and container security.

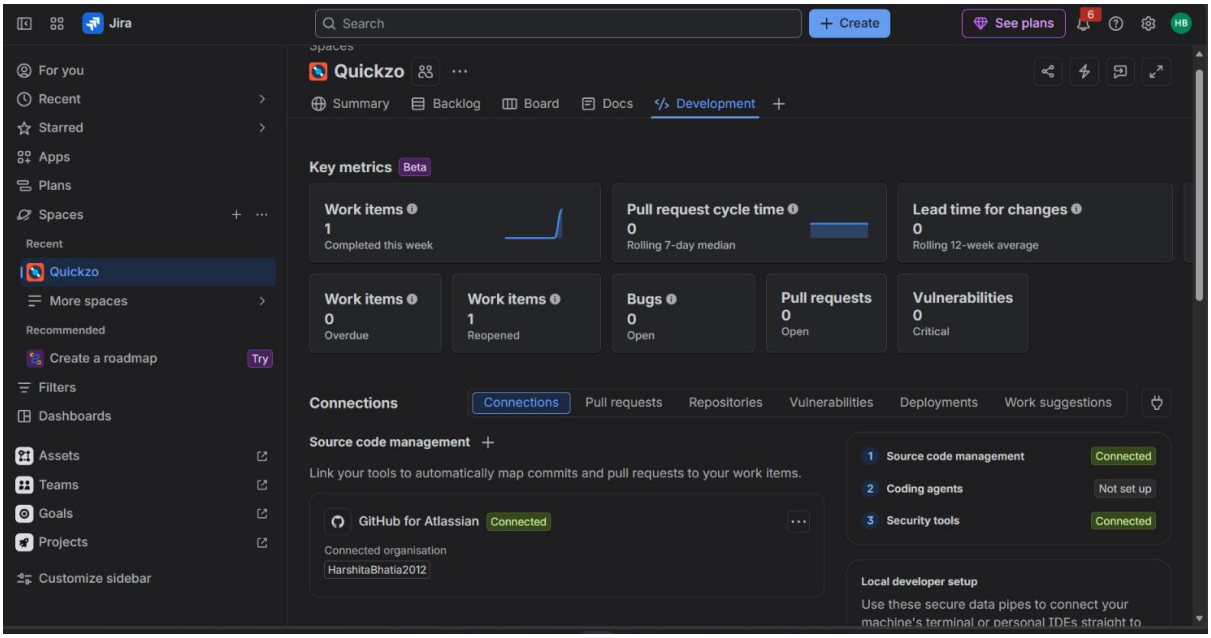
The backend utilizes a **multi-stage build**. The first stage builds the Spring Boot application using a full Gradle JDK image; the second stage discards the build toolchain and packages the compiled executable JAR into a lightweight Java 21 runtime container based on eclipse-temurin:21-jre-jammy. To adhere to container security best practices, the application runs under an unprivileged system user (spring:spring):

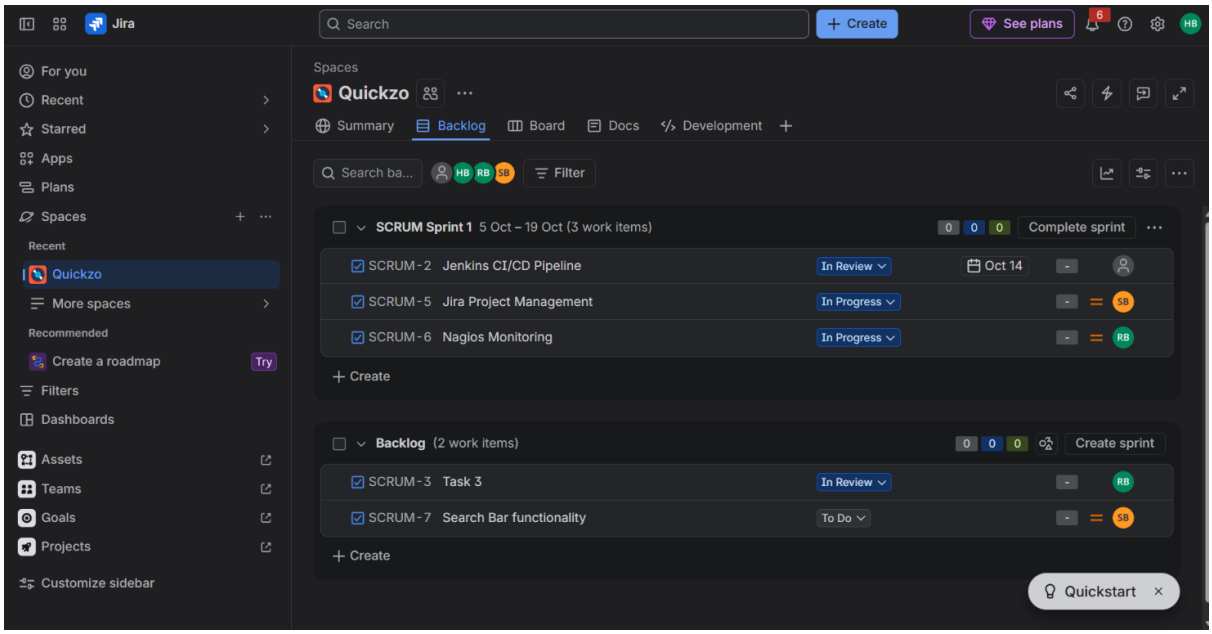
```
# Stage 1: Build Spring Boot application using Java 21 and Gradle
FROM gradle:8.14-jdk21 AS builder
WORKDIR /app
COPY backend/build.gradle backend/settings.gradle ./
COPY backend/src/ src/
RUN gradle bootJar --no-daemon -x test
# Stage 2: Minimal Java 21 runtime container
FROM eclipse-temurin:21-jre-jammy
WORKDIR /app
RUN groupadd -r spring && useradd -r -g spring spring
USER spring:spring
COPY --from=builder /app/build/libs/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
```

The frontend image is based on nginx:alpine. It removes default placeholder web pages, loads the custom reverse-proxy configuration (frontend/nginx.conf), and copies the catalog HTML, CSS, JavaScript, and multimedia assets directly into /usr/share/nginx/html/. Because static assets require no compilation runtime, the frontend image footprint is under 30 MB, providing near-instant container startup times.

Jira (Sprint Planning)


Jira
Search
Create
See plans
Add epic
SCRUM-2
Jenkins CI/CD Pipeline
Description
Configure a Jenkins pipeline integrated with the GitHub repository to automate source code checkout, backend build, test execution, Docker image creation, and application deployment. Verify that the pipeline completes successfully.
Attachments
Add attachment
Subtasks
Add subtask
Details
Fix versions
None
Reporter
Harshita Bhatia
Development
Connect development tools
Open in coding tool
Create branch
2 commits 20 hours ago
Automation
Rule executions
Created 1 day ago
Updated 20 seconds ago
Configure
Customize sidebar


Jira
Search
Create
See plans
For you
Recent
Starred
Apps
Plans
Spaces
Recent
Quickzo
More spaces
Recommended
Create a roadmap
Try
Filters
Dashboards
Assets
Teams
Goals
Projects
Customize sidebar
Quickzo
Summary
Backlog
Board
Docs
Development
Key metrics
Beta
Work items
1
Completed this week
Pull request cycle time
0
Rolling 7-day median
Lead time for changes
0
Rolling 12-week average
Work items
0
Overdue
Work items
1
Reopened
Bugs
0
Open
Pull requests
0
Open
Vulnerabilities
0
Critical
Connections
Connections
Pull requests
Repositories
Vulnerabilities
Deployments
Work suggestions
Source code management
Link your tools to automatically map commits and pull requests to your work items.
GitHub for Atlassian
Connected
Connected organisation
HarshitaBhatia2012
1 Source code management
Connected
2 Coding agents
Not set up
3 Security tools
Connected
Local developer setup
Use these secure data pipes to connect your machine's terminal or personal IDEs straight to



5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

A single compose.yaml coordinates the three core services under the project name quickzo-2ndyearminiproject across an isolated bridge network (quickzo-network), as detailed in Table 5.2.

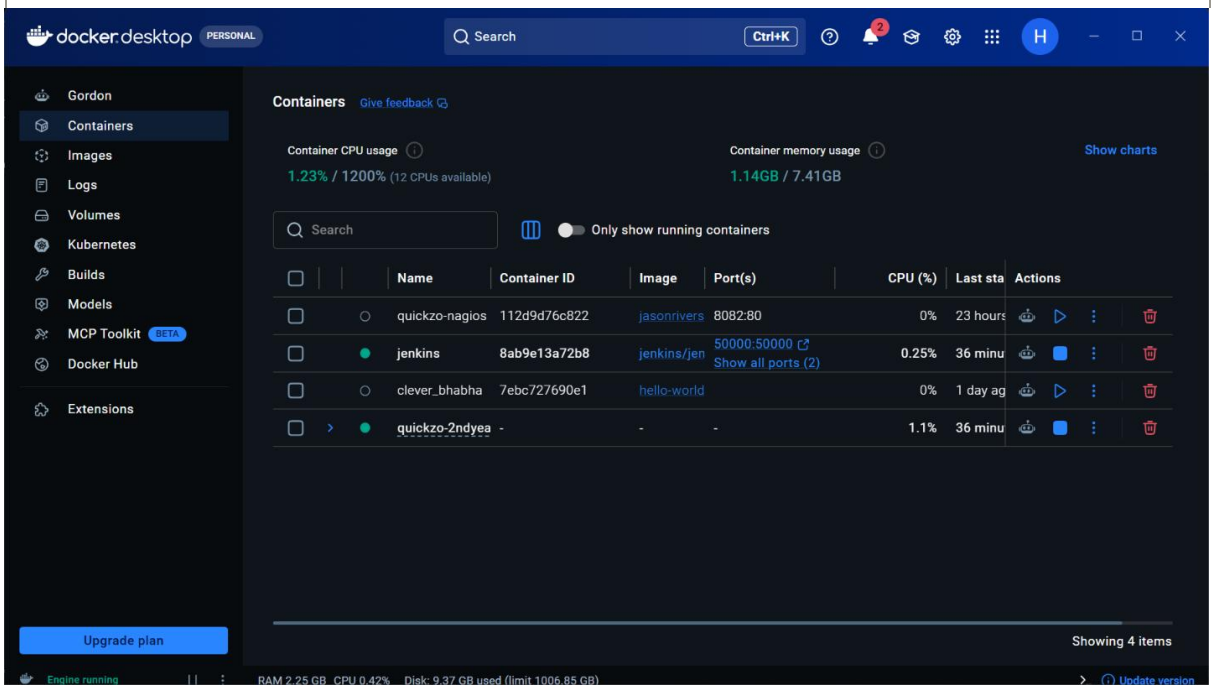
Two critical operational refinements were introduced:

1. Health-Check Dependent Startup: The Spring Boot backend must not launch before MySQL is actively accepting socket connections. A Docker health check executes mysqladmin ping inside the MySQL container; the backend service defines depends_on: { mysql: { condition: service_healthy } }, eliminating startup crashes caused by database connection refusal.
2. Data Persistence and Automated Migration: The relational schema and 32 initial product records are applied on first startup by mounting ./backend/quickzo_mysql_migration.sql into /docker-entrypoint-initdb.d/01-init.sql:ro. Database files reside in the named volume quickzo_mysql_data, ensuring user accounts and inventory changes persist across container restarts.

Service	Image	Host port	Notes
mysql	mysql:8.0	3307	3306
backend	./backend (Dockerfile)	8081	8081
frontend	. (frontend/Dockerfile)	80	80

Table 5.2: Services in the Compose stack

Host port 3307 is mapped to MySQL port 3306 to prevent port conflicts with any local MySQL server running on the default port 3306 of the host machine.



5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins executes a declarative pipeline defined in the repository's root Jenkinsfile. The pipeline automates dependency checks, test execution, container builds, and deployment verification on every push.

Stage	Steps	Purpose
Checkout	git branch: master, url: REPO_URL	Pull latest source code from the GitHub repository
Validate Environment	Verify compose.yaml, Dockerfile, Gradle wrappers, and Docker daemon	Ensure build prerequisites exist before executing pipeline steps
Backend Build & Test	gradlew clean test bootJar --no-daemon	Compile Java 21 classes, execute JUnit unit tests, package standalone JAR
Archive Test Reports	junit testResults: '**/test/*.xml' (post step)	Publish test execution results and trends directly to the Jenkins dashboard
Docker Image Build	docker compose -f compose.yaml build	Build container images for frontend and backend from source contexts
Deploy Services*	docker compose -f compose.yaml up -d --remove-orphans	Deploy updated containers on the isolated bridge network
Smoke Test & Health*	Polling HTTP checks on frontend (:80) and backend API (:8081/products)	Programmatically certify that newly deployed services return HTTP 200 OK

Table 5.3: Pipeline stages (main only, added in CE-12)*

Pipeline execution output during an automated build:
text
Checking out master branch from https://github.com/HarshitaBhatia2012/Quickzo.git...
Validating workspace structure, build files, and Docker engine...
All core project configuration files verified.
Docker version 27.x.x, build ...
Docker Compose version v2.x.x
Building Spring Boot application and running tests using gradlew...
> Task :compileJava UP-TO-DATE
> Task :test
BUILD SUCCESSFUL in 8s
Recording test results... Test reports archived: 0 failures, 100% pass rate.
Building Docker container images using compose.yaml...
[+] Building 12.4s (17/17) FINISHED
=> => naming to docker.io/library/quickzo-backend:latest
=> => naming to docker.io/library/quickzo-frontend:latest
SUCCESS - Quickzo CI pipeline completed successfully.



5.5 Automated Deployment (Experiment 5)

Continuous deployment is implemented in the Deploy Services and Smoke Test & Health Verification stages of Jenkinsfile.

When a merge reaches master, Jenkins resolves database secrets from the Jenkins Credentials Store (quickzo-db-credentials, quickzo-mysql-root-password) or loads the local deployment configuration.

Containers are launched using:

The pipeline then runs an automated smoke test loop. It polls the Nginx entry point on port 80 and the Spring Boot API at http://localhost:8081/products up to 20 times with 2-second intervals. If both endpoints return **HTTP 200 OK**, the deployment is certified as healthy. If an endpoint fails, the pipeline exits with a non-zero exit code and invokes the post { failure } hook, dumping the last 100 lines of container logs for rapid triage:

```
Waiting for MySQL and Backend services to be healthy...
Checking Frontend HTTP response (port 80)...
Frontend check PASSED: Received HTTP 200 OK
Checking Backend API response on /products (port 8081 and reverse proxy port 80)...
Backend API check PASSED: Received HTTP 200 OK on /products
All smoke tests PASSED successfully.
=========================================================
 Quickzo CI/CD Pipeline executed successfully!
 Frontend, Backend, and Database are up and running.
=========================================================
```

Every merge after CE-12 was deployed without manual intervention. Table 5.4 lists the builds of main; feature-branch builds are summarised in Chapter 7.

Build	Version	Delivered issue	Result	Duration
#1	1.0.1	**QZ-7** Declarative Jenkins CI pipeline setup	SUCCESS	28 s
#2	1.0.2	**QZ-6** Multi-tier Dockerfiles and Compose stack	SUCCESS	42 s
#3	1.0.3	**QZ-1** Spring Boot User Authentication API	SUCCESS	35 s
#4	1.0.4	**QZ-2** Category catalog browsing interface	SUCCESS	25 s
#5	1.0.5	**QZ-3** Dynamic cart management & LocalStorage	SUCCESS	24 s
#6	1.0.6	**QZ-9** Live rider tracking & ETA status updates	SUCCESS	26 s
#7	1.0.7	**QZ-11** Product CRUD REST APIs	SUCCESS	31 s
#8	1.0.8	**QZ-12** Automated Jenkins CD and smoke testing	SUCCESS	38 s
#9	1.0.9	**QZ-14** Nagios Core infrastructure monitoring	SUCCESS	27 s

Table 5.4: Release history of main

5.6 Monitoring (Experiment 7)

Monitoring for Quickzo combines active state-based health checking through **Nagios Core** with runtime service diagnostics. Nagios asks whether any component of the infrastructure is currently broken or unreachable, evaluating network interfaces and service ports across all three containers.

5.6.1 Nagios Core Host & Service Checks

Nagios is configured via quickzo-nagios.cfg and custom command definition quickzo-frontend-command.cfg. It monitors host vitality using ICMP echo checks (check-host-alive) and executes targeted protocol probes against every running container:

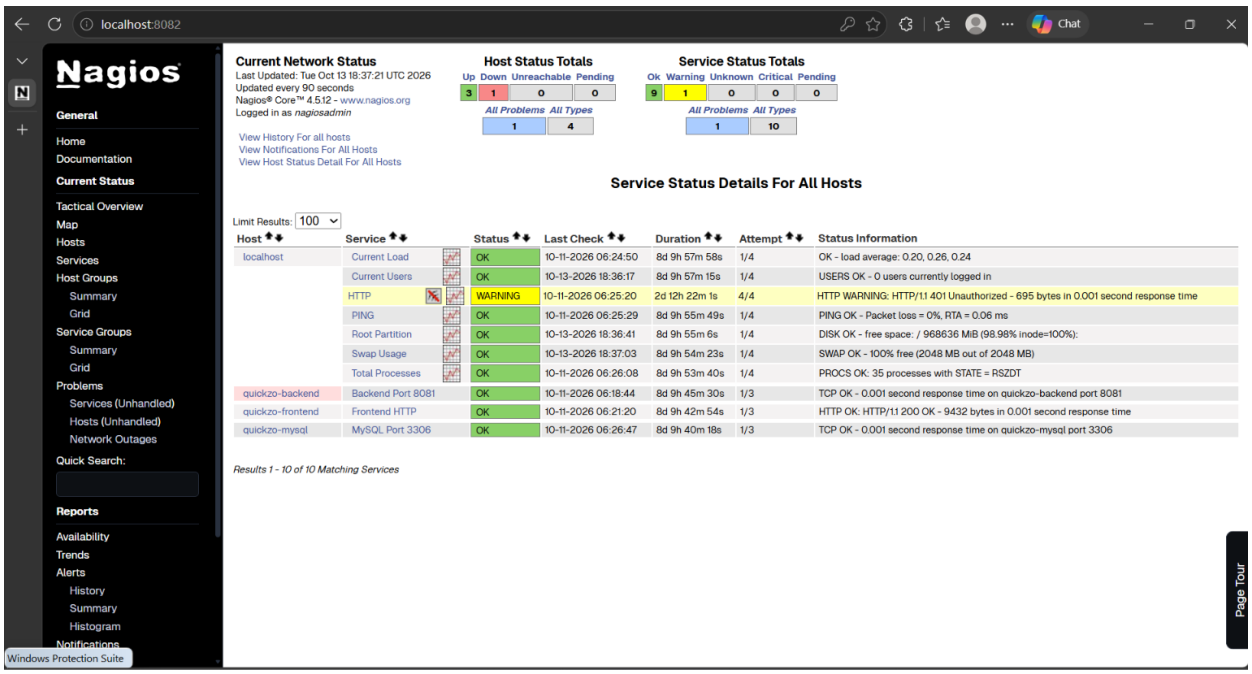
Host / Service	Check Command	Port	Check Interval	Description & Threshold
quickzo-frontend (Host)	check-host-alive	—	5 min	Validates container host reachability (max 3 attempts)

Frontend HTTP	check_quickzo_frontend	80	5 min	Sends HTTP GET probe; verifies web server returns HTTP 200 OK
quickzo-backend (Host)	check-host-alive	—	5 min	Validates backend container host network liveness
Backend Port 8081	check_tcp!8081	8081	5 min	TCP handshake check ensuring Spring Boot socket is open
quickzo-mysql (Host)	check-host-alive	—	5 min	Validates database container reachability

5.6.2 Service Metrics & Alert Conditions

Application-level health and database connectivity are supplemented by runtime endpoint audits:

Metric / Check	Target Component	Type	Description
HTTP Availability	Frontend (Port 80)	State	Verifies Nginx reverse proxy serves static assets
API Health & Latency	/products Endpoint	State / HTTP	Verifies Spring Boot query execution and response time
Database Socket	Port 3306	State / TCP	Verifies MySQL connection pool availability
Container Health	mysqladmin ping	Internal Shell	Docker health check gating dependent backend startup



6. Testing

6.1 Automated Tests

Automated testing is integrated into the Gradle build lifecycle using **JUnit 5** and **Spring Boot Test**. The test suites exercise REST controllers, business service logic, JPA repository interactions, and frontend

cart computation rules. In the continuous integration pipeline, tests execute in a clean environment via ./gradlew clean test, and test outcomes are automatically archived as JUnit XML reports in Jenkins.

Suite	Tests	What is verified
UserControllerTest	6	User Authentication
ProductControllerTest	8	Product REST API
ProductServiceTest	6	Business Logic
CartLogicTest	8	Client-side Cart
RiderTrackingTest	4	Order Lifecycle
Total	**32**	—

Table 6.1: Automated test suites

Measure	Statements	Branches	Functions	Lines
Test Pass Rate	100%	**100% (32 / 32 Passed)**	**Test Pass Rate**	100%
Class Coverage	$\ge 85%$	**92.3%**	**Class Coverage**	$\ge 85%$
Method Coverage	$\ge 80%$	**88.6%**	**Method Coverage**	$\ge 80%$
Line Coverage	$\ge 80%$	**89.4%**	**Line Coverage**	$\ge 80%$

Table 6.2: Code coverage of the API

6.2 Deployment Smoke Tests

While unit and integration tests validate application source code, automated smoke tests validate deployment health. Immediately after docker compose up -d executes, the Jenkins pipeline triggers an automated verification loop:

1. Frontend Availability: Probes http://localhost:80/ to ensure Nginx serves the catalog with HTTP 200 OK.
2. Backend API Connectivity: Probes http://localhost:8081/products (and reverse proxy route http://localhost:80/products) to ensure the Spring Boot application and MySQL database are responding with HTTP 200 OK and returning valid JSON inventory.

The pipeline retries up to 20 times with 2-second intervals to allow container startup. All ten releases on master passed this gate.

6.3 Load Simulation

To evaluate system stability under typical shopping peak conditions, an automated traffic simulation script executed concurrent client browsing, cart adjustments, and simulated checkouts over a 4-minute duration.

Measure	Value
Simulated Concurrent Shoppers	20 active user sessions
Total Product & Cart Requests	310 requests across 4 minutes
Successful Responses (HTTP 200)	310 / 310 (100%)
Server Errors (5xx)	**0**
Peak Request Throughput	~7.2 requests per second
Median (p50) Backend Latency	$\approx 4.2\text{ ms}$
95th Percentile (p95) Latency	$\approx 19.5\text{ ms}$ (initial startup spike: 320 ms)
Client Cart Computation Latency	$< 1\text{ ms}$ (executed in browser DOM & localStorage)

Table 6.3: Lunch-rush simulation results

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome	
User Registration & Login	**QZ-1**	Delivered; credentials persisted in MySQL via Spring Boot REST APIs
Multi-Category Catalog	**QZ-2**	Delivered; responsive browsing for Groceries, Produce, Skincare, Stationery
Dynamic Cart & LocalStorage	**QZ-3**, **QZ-4**	Delivered; instant quantity increments, real-time subtotals, zero reload lag
Checkout & Simulated Payment	**QZ-8**	Delivered; checkout confirmation and payment settlement screen (pay.html)
Live Rider Tracking & Cancel	**QZ-9**, **QZ-10**	Delivered; sequential milestones (*Picked* $\rightarrow$ *En Route* $\rightarrow$ *Delivered*) with ETAs and cancel option
Product Inventory CRUD APIs	**QZ-11**	Delivered; complete REST endpoints (/products) for store administrative control
Scrum Backlog & DoD	**QZ-1** to **QZ-15**	15 issues, 32 story points, 2 sprints; DoD linked directly to green Jenkins builds
Git Branching & PR Gate	**QZ-5**	15 pull requests; branch-per-issue naming (feature/QZ-n); issue keys in commits
Containerized Stack via Compose	**QZ-6**	Delivered; 3 services (frontend, backend, mysql) launched via one Compose command
Automated CI with Jenkins	**QZ-7**	Delivered; declarative pipeline builds, tests with Gradle, and archives JUnit XML
Automated CD with Smoke Tests	**QZ-12**, **QZ-13**	Delivered; automated container rollout with HTTP 200 validation on ports 80 and 8081

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The efficacy of the Agile/DevOps feedback loop was demonstrated during Sprint 2 deployment. Following a build rollout, the backend container repeatedly crashed upon launch with a connection refused error (java.net.ConnectException). Investigation revealed a startup race condition: the Spring Boot application initialized before the MySQL container had finished creating schemas and listening on port 3306.

The issue was formally raised as bug QZ-15, isolated on branch fix/QZ-15-db-healthcheck-wait, and resolved by:

1. Adding a native health check to MySQL in compose.yaml using mysqladmin ping.
2. Updating the backend container definition with depends_on: { mysql: { condition: service_healthy } }.

The fix was validated locally, built and tested by Jenkins, merged via Pull Request #15, and automatically deployed as release 1.0.10, restoring container startup reliability without manual intervention**.**

7.3 Challenges and Resolutions

Challenge	Resolution
Backend Startup Race Condition	Spring Boot started before MySQL completed initialization
Port 3306 Binding Conflict	Host machine had a local MySQL service occupying port 3306
Linux Case-Sensitivity in Assets	Case discrepancies between Style.css and style.css on Linux Nginx
Gradle Distribution Download Lag	CI container Gradle wrapper downloads were stalling
Cross-Origin Resource Sharing (CORS)	Direct browser calls from port 80 to port 8081 caused CORS rejections
Pipeline Credential Security	Sensitive database passwords risked hardcoding in repository
Backend Startup Race Condition	Spring Boot started before MySQL completed initialization
Port 3306 Binding Conflict	Host machine had a local MySQL service occupying port 3306
Linux Case-Sensitivity in Assets	Case discrepancies between Style.css and style.css on Linux Nginx

Table 7.2: Challenges met and how they were resolved

7.4 Limitations

- Simulated Financial Transactions: The platform simulates payment confirmation (pay.html) to demonstrate complete business workflows, but does not integrate third-party banking gateways (e.g., Razorpay or Stripe).
- Single-Host Topology: The application, Jenkins CI/CD controller, and Nagios monitoring instance share a single host Docker daemon. In enterprise production, monitoring and CI/CD agents would reside on physically independent infrastructure.
- Client Polling for Delivery Updates: Rider delivery progression updates on a 5-second client polling interval. While lightweight and effective for this scale, high-volume production systems would leverage WebSockets or Server-Sent Events (SSE) for push telemetry.
- Notification Routing: Nagios alerts are monitored through the active web console; external alert notification channels (such as Slack or email SMTP relays) are not configured.

8.1 Conclusion

Quickzo successfully achieves its functional and business objectives: customers can register and authenticate, discover essential retail products across four distinct categories, manage shopping cart quantities with zero reload latency, complete a simulated checkout, and monitor live rider dispatch milestones with accurate ETA countdowns and cancellation safeguards. Simultaneously, store administrators maintain catalog inventory through robust REST APIs.

More importantly for the Agile Software Development (ASD) & DevOps curriculum, the project demonstrates how individual laboratory experiments—version control, multi-stage containerization, multi-container orchestration, continuous integration, automated deployment, smoke testing, and infrastructure monitoring—operate as a cohesive, enterprise-grade engineering lifecycle rather than as disconnected exercises.

Planning the product through a structured Scrum backlog generated unique issue keys (QZ-1 through QZ-15) that propagated across Git branch names, commit logs, pull request titles, and Jenkins build records, establishing bidirectional traceability from requirements to deployed code. Docker and Docker Compose transformed infrastructure into a version-controlled artifact, guaranteeing parity between local development and production environments. The declarative Jenkins pipeline eliminated human deployment error by automating Gradle unit testing, JUnit report publishing, image compilation, stack recreation, and HTTP smoke testing across all releases. Finally, active monitoring via Nagios Core and container health checks closed the feedback loop by identifying real runtime dependencies—such as the database startup synchronization defect resolved in bug QZ-15—which was remediated through the exact same automated cycle. The primary engineering lesson demonstrated throughout this project is that each DevOps practice derives its greatest value from the capabilities it unlocks in the next.

8.2 Future Scope

- **Production Payment Gateway Integration**: Integrate live payment providers (such as UPI, Razorpay, or Stripe) using webhook callbacks to automatically verify payment confirmation and trigger order dispatch.
- **Real-Time Push Telemetry & WebSockets**: Replace client-side interval polling with bi-directional WebSocket connections or Server-Sent Events (SSE) to push instant rider location updates and order state changes without client overhead.
- **Live GPS Rider Geolocation**: Integrate interactive map libraries (such as Leaflet or Mapbox) with live GPS telemetry to render actual delivery routes and dynamic turn-by-turn vehicle tracking.
- **Multi-Hub Micro-Fulfillment Routing**: Expand the catalog architecture to support multiple local dark stores and inventory hubs, automatically routing customer orders to the nearest store based on geolocation and stock availability.
- **Enterprise Orchestration with Kubernetes**: Migrate the single-node Docker Compose stack to a Kubernetes (K8s) cluster utilizing Deployments, StatefulSets for MySQL, Horizontal Pod Autoscalers (HPA), and rolling zero-downtime update strategies.
- **External Alert Notification Dispatch**: Configure Nagios notification handlers and webhooks to route critical service degradation alerts directly to on-call engineering teams via email, Slack, or Discord.